

README – Azure Container Apps + MCP + OpenAPI (Cloud Shell Runbook)

Azure Container Apps + MCP + OpenAPI (Cloud Shell Runbook)

This document is a **complete, reproducible guide** to deploy an **Employee API** and an **MCP Server** on **Azure Container Apps**, build images using **Azure Container Registry (ACR Tasks)**, and generate an **OpenAPI** (`openapi.json`) specification for use with **Azure AI Foundry agents** — **entirely from Azure Cloud Shell**.

Architecture Overview



Prerequisites

- Azure subscription with permission to create:
 - Resource Groups
 - Azure Container Registry
 - Azure Container Apps
- GitHub repository containing:
 - `employee-api/` (Dockerized backend API)
 - `mcp-server/` (Dockerized MCP server)
 - `mcp-server/generate_openapi.py`
- Azure Portal access (for Azure AI Foundry)

All commands below are executed from **Azure Cloud Shell (Bash)** unless specified.

1. Open Azure Cloud Shell

1. Open Azure Portal
 2. Click **Cloud Shell** (>_)
 3. Select **Bash**
 4. Allow Cloud Shell to initialize storage (~/clouddrive)
-

2. Define Environment Variables

```
1 RG="employee-mcp-rg"
2 LOCATION="eastus"
3 ACR="employee-mcp-acr931"
4 ENV="employee-mcp-env"
5
```

Shell ▾

3. Create Resource Group

```
1 az group create \
2   --name "$RG" \
3   --location "$LOCATION"
4
```

Shell ▾

4. Create Azure Container Registry (ACR)

```
1 az acr create \
2   --resource-group "$RG" \
3   --name "$ACR" \
4   --sku Basic \
5   --admin-enabled true
6
```

Shell ▾

Verify:

```
1 az acr show -n "$ACR" -o table
2
```

Shell ▾

5. Clone the Project Repository

Shell ▾

```
1 cd ~/clouddrive
2 git clone https://github.com/<your-username>/foundry-agent
3 cd foundry-agent
4
```

Verify structure:

Shell ▾

```
1 ls
2 # employee-api/
3 # mcp-server/
4
```

6. Build and Push Images to ACR (ACR Tasks)

6.1 Build employee-api

Shell ▾

```
1 az acr build \
2   --registry "$ACR" \
3   --image employee-api:latest \
4   ./employee-api
5
```

6.2 Build mcp-server

Shell ▾

```
1 az acr build \
2   --registry "$ACR" \
3   --image mcp-server:latest \
4   ./mcp-server
5
```

Verify images:

Shell ▾

```
1 az acr repository list --name "$ACR" -o table
2 az acr repository show-tags --name "$ACR" --repository employee-api -o table
3 az acr repository show-tags --name "$ACR" --repository mcp-server -o table
4
```

7. Create Container Apps Environment

Shell ▾

```
1 az containerapp env create \  
2   --name "$ENV" \  
3   --resource-group "$RG" \  
4   --location "$LOCATION" \  
5
```

8. Retrieve ACR Credentials (for image pull)

Shell ▾

```
1 ACR_LOGIN_SERVER=$(az acr show -n $ACR --query loginServer -o tsv) \  
2 ACR_USERNAME=$(az acr credential show -n $ACR --query username -o tsv) \  
3 ACR_PASSWORD=$(az acr credential show -n $ACR --query "passwords[0].value" -o tsv) \  
4
```

9. Deploy employee-api Container App

Shell ▾

```
1 az containerapp create \  
2   --name employee-api \  
3   --resource-group "$RG" \  
4   --environment "$ENV" \  
5   --image "$ACR_LOGIN_SERVER/employee-api:latest" \  
6   --target-port 9000 \  
7   --ingress external \  
8   --registry-server "$ACR_LOGIN_SERVER" \  
9   --registry-username "$ACR_USERNAME" \  
10  --registry-password "$ACR_PASSWORD" \  
11
```

Get backend URL:

Shell ▾

```
1 BACKEND_URL=$(az containerapp show \  
2   --name employee-api \  
3   --resource-group "$RG" \  
4   --query properties.configuration.ingress.fqdn -o tsv) \  
5 \  
6 echo "Backend API: https://$BACKEND_URL" \  
7
```

10. Deploy mcp-server Container App

Shell ▾

```
1 az containerapp create \  
2   --name mcp-server \  
3   --resource-group "$RG" \  
4   --environment "$ENV" \  
5   --image "$ACR_LOGIN_SERVER/mcp-server:latest" \  
6   --target-port 9000 \  
7   --ingress external \  
8   --registry-server "$ACR_LOGIN_SERVER" \  
9   --registry-username "$ACR_USERNAME" \  
10  --registry-password "$ACR_PASSWORD" \  
11  --env-vars BASE_URL="https://$BACKEND_URL" MCP_API_KEY="dev_secret_key_123" \  
12
```

Get MCP URL:

Shell ▾

```
1 MCP_URL=$(az containerapp show \  
2   --name mcp-server \  
3   --resource-group "$RG" \  
4   --query properties.configuration.ingress.fqdn -o tsv) \  
5 \  
6 echo "MCP Server: https://$MCP_URL" \  
7
```

11. Update MCP Server With Its Own URL

Shell ▾

```
1 az containerapp update \  
2   --name mcp-server \  
3   --resource-group "$RG" \  
4   --set-env-vars MCP_SERVER_URL="https://$MCP_URL" \  
5
```

12. Install Python Dependencies for OpenAPI Generation

Cloud Shell Python is minimal; install dependencies locally:

Shell ▾

```
1 pip install --user -r mcp-server/requirements.txt \  
2
```

13. Generate openapi.json

Shell ▾

```
1 export MCP_SERVER_URL="https://$MCP_URL"
2 python mcp-server/generate_openapi.py
3
```

Verify:

Shell ▾

```
1 ls openapi.json
2
```

14. View or Copy openapi.json

- View:

Shell ▾

```
1 less openapi.json
2
```

- Open in editor:

Shell ▾

```
1 code openapi.json
2
```

- Download:

- Cloud Shell toolbar → Upload/Download files → Download `openapi.json`

15. Add OpenAPI Action in Azure AI Foundry

1. Open Azure AI Foundry
2. Open your Project → Agents
3. Create or open an agent
4. Add Tool → OpenAPI
5. Upload `openapi.json`
6. Set:

- Base URL: `https://<mcp-server-fqdn>`
- Auth: None

16. Test the Agent

Example prompt:

"Add a new employee named Varun"

Validation path:

1 Foundry Agent → OpenAPI Action → MCP Server → employee-api
2

Shell ▾

Logs can be viewed via **Azure Portal** → **Container App** → **Log stream**.

Final Outcome

-  Images built using ACR Tasks
 -  Services running on Azure Container Apps
 -  MCP server exposed publicly
 -  OpenAPI spec generated from live infra
 -  Azure AI Foundry agent invoking real APIs
-

Notes & Next Steps (Optional)

- Replace ACR admin credentials with **Managed Identity + AcrPull**
 - Add authentication to MCP endpoints
 - Lock down public ingress for production
-

This README is intentionally explicit and reproducible. If you clone the repo and follow it top-to-bottom, you will reach the same working state.